

说明

ARGB 外设通过 DMA 实现硬件时序生成，提供低 CPU 占用、高时序精准度和低内存开销，支持多种灯控协议（如 WS2812、SK6812 等），适用于大规模灯光系统、电竞灯效、汽车灯光和音乐可视化等场景，实现稳定、同步且高效的灯光控制。

适用范围

适用范围	系列
通用 MCU	CH32X315、CH32V407 等带有 ARGB 外设的通用 MCU

目录

说明 i

目录 ii

表格索引 ii

第 1 章 ARGB 优势 1

 1.1 WS2812 主流控制方案对比 1

 1.2 ARGB 的主要优势 1

第 2 章 使用方法 3

 2.1 软件开发流程 3

 2.2 典型应用场景 5

历史版本 6

声明 7

表格索引

表 1-1 WS2812 主流控制方案对比 1

第1章 ARGB 优势

1.1 WS2812 主流控制方案对比

表 1-1 WS2812 主流控制方案对比

对比维度	TIM(PWM+DMA)	ARGB	SPI + DMA	PIOC
内存开销	高（单个 bit 须转换成 CVR 或 CVR/2 宽度的数据，数据膨胀 8-16 倍）	低（直接发送原始 RGB 数据，无需波形扩展）	高（需要额外的缓冲区存放转换后的 SPI 波形数据，通常是原始数据的 8 倍）	低（直接发送原始 RGB 数据，无需波形扩展）
CPU 占用率	中（需要配置 DMA，在每次发送前需要额外转换数据）	低（无需额外转换数据）	中（同 TIM，依赖 DMA 且需转换数据）	零（主 CPU 下发任务后即可休眠或处理其他业务，PIOC 独立运行）
时序精准度	高（硬件生成波形，精度取决于 PB 时钟）	高（ARGB 挂载在 HB 时钟下，精度直接取决于 HB 时钟）	中（依赖 SPI 波特率，但受限于时钟分频，有时难以完美凑出 WS2812 的特定纳秒值）	高（PIOC 运行在 HB 时钟下，指令周期确定，可编写汇编级精确时序）
代码复杂度	中需配置定时器、DMA 通道，计算 ARR/PSC、数据预处理	低（需配置 ARGB、DMA）	中需配置 SPI、DMA 通道，计算分频系数、数据预处理	高（WCH 提供标准库封装，底层逻辑固化在 PIOC ROM，应用层调用简单，但自定义时序时需要编写汇编代码）
灵活性	低（修改时序需重算参数或修改缓冲数据生成算法）	高（ARGB 不可编程，灵活程度不如 PIOC，但绝大多数参数可配置，只支持数据输出）	中（改时序需改波特率或缓冲数据生成算法）	极高（PIOC 是可编程的，不仅支持 WS2812，还可自定义其他私有协议，数据不仅可以输出，还可以输入）

1.2 ARGB 的主要优势

抗中断干扰：一旦启动，波形生成完全独立于 CPU。即使主频满载、开启高优先级中断（如 USB、电机控制），输出波形也绝无抖动或错位，彻底杜绝了软件模拟导致的“花屏”现象。

使用方便，开发快速直观：**相较于 SPI+DMA 或 TIM+DMA 的方案，ARGB 无需在每次刷新之前对数据做预处理，数据排列方式可以和 WS2812 的要求一致（GRB 排列）。无需编写 PIOC 的微型协处理器的协议栈代码，编写 PIOC 的代码以及调试相对繁琐，对快速开发不友好。

没有“数据膨胀”：SPI+DMA 或 TIM+DMA 的方案需要用大于至少三倍的内存开销用来控制控制器时序的产生。对于长灯带，例如 1000 个灯珠，使用常规的 SPI+DMA 或 TIM+DMA 内存开销会从 3K 暴增到 24K，这对 RAM 容量较小的 MCU 是致命的。

极高的协议兼容性：**参数完全可配，不同于 SPI 固定的占空比，ARGB 外设允许用户自由配置：0 码/1 码的高电平宽度 (T0H, T1H)，0 码/1 码的总周期 (DAT_ARR)，复位信号宽度 (RST_ARR)。通过调整上述参数，一套硬件电路即可完美兼容 WS2812, SK6812 (RGBW), TM1814, UCS1903,

GS8208 等不同厂商、不同时序要求的灯控 IC，无需更换硬件或重写底层驱动逻辑。

零 CPU 占用与 DMA 高效传输：解放算力，**在长灯带（如 1000+ 颗灯珠）刷新时，CPU 占用率接近 0%。释放出的宝贵算力可用于运行复杂的灯光算法（如音乐频谱分析、AI 视觉处理）或多任务实时控制。

灵活的刷新机制与中断管理：提供发送完成、半传输、字节完成、复位完成等多种中断，方便实现双缓冲切换、动态效果更新等高级功能。

第2章 使用方法

2.1 软件开发流程

1. 初始化 IO
2. 初始化 DMA
3. 初始化 RGB
4. 使能 ARGB 发送使能 ARGB_Cmd(ENABLE);开始发送一帧数据

根据不同芯片，配置 ARGB 相应的 IO 为复用推挽模式，具体引脚功能可以参考官网中的[数据手册](#)，例如在 CH32V407 系列中，ARGB 的复用引脚是 PA15。CH32V407 配置代码如下：

```
void ARGB_GPIO_Init()
{
    GPIO_InitTypeDef GPIO_InitStructure;

    RCC_PB2PeriphClockCmd(RCC_PB2Periph_GPIOA, ENABLE);
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_15;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_High;
    GPIO_Init(GPIOA, &GPIO_InitStructure);
}
```

根据不同芯片，配置 DMA 通道，不同的芯片的 ARGB 可能有不同的与之绑定的 DMA 通道。DMA 的传输模式推荐固定为 DMA_Mode_Normal，DMA 的传输长度可按照需求配置，但是需要注意的是：DMA 的传输长度须严格与 ARGB 的数据刷新周期长度一致。CH32V407 配置代码如下：

```
void ARGB_WS2812B_DMA_Init(uint8_t *ARGB_DMA_Buffer, size_t buffer_size)
{
    RCC_HBPeriphClockCmd(RCC_HBPeriph_DMA1, ENABLE);

    DMA_InitTypeDef DMA_InitStructure;
    DMA_InitStructure.DMA_PeripheralBaseAddr = (uint32_t)&(ARGB->DATAR);
    DMA_InitStructure.DMA_MemoryBaseAddr = (uint32_t)ARGB_DMA_Buffer;
    DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralDST;
    DMA_InitStructure.DMA_BufferSize = buffer_size;
    DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
    DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
    DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Byte;
    DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_Byte;
    DMA_InitStructure.DMA_Mode = DMA_Mode_Normal;
    DMA_InitStructure.DMA_Priority = DMA_Priority_High;
    DMA_InitStructure.DMA_M2M = DMA_M2M_Disable;
    DMA_Init(DMA1_Channel10, &DMA_InitStructure);
    DMA_Cmd(DMA1_Channel10, ENABLE);
}
```

在配置 ARGB 时需要注意单线协议的参数如 T1H 是根据 ARGB 挂载的时钟总线的时钟来计算的，例如在 CH32V407 系列中，ARGB 是在 HB 时钟总线下的，该系列的 HB 总线时钟是与内核时钟相同的，故可以直接使用 SystemCoreClock 来计算应写入寄存器的值，否则，根据系统时间结构或者直接调用 void RCC_GetClocksFreq(RCC_ClocksTypeDef *RCC_Clocks); 函数来获取当前平台下 ARGB 所在时钟总线的时钟，再去计算应写入寄存器的值。

```

#define ARGB_MS2HBTICK(ms) ((uint32_t)((float)SystemCoreClock / 1000.0f * (float)ms))
#define ARGB_US2HBTICK(us) ((uint32_t)((float)SystemCoreClock / 1000000.0f * (float)us))
#define ARGB_NS2HBTICK(ns) ((uint32_t)((float)SystemCoreClock / 1000000000.0f * (float)ns))
void ARGB_WS2812B_Init(uint8_t *ARGB_DMA_Buffer, size_t buffer_size)
{
    // Enable the clock for ARGB peripheral
    RCC_HBPeriphClockCmd(RCC_HBPeriph_ARGB, ENABLE);

    // Initialize ARGB configuration structure
    ARGB_InitTypeDef ARGB_InitStructure;
    // Configure ARGB parameters
    ARGB_InitStructure.ARGB_Length = buffer_size - 1; // Set buffer length (minus 1 for zero-based indexing)
    ARGB_InitStructure.ARGB_T1H = ARGB_US2HBTICK(0.9f); // Set high time for '1' bit
    ARGB_InitStructure.ARGB_T0H = ARGB_US2HBTICK(0.3f); // Set high time for '0' bit
    ARGB_InitStructure.ARGB_DataPeriod = ARGB_US2HBTICK(1.45f); // Set data bit period
    ARGB_InitStructure.ARGB_RSTPeriod = ARGB_US2HBTICK(80.0f); // Set reset period
    ARGB_InitStructure.ARGB_Mode = ARGB_Mode_SendRSTFirst; // Set mode to send reset first
    ARGB_InitStructure.ARGB_Endian = ARGB_Endian_MSB; // Set most significant bit first
    ARGB_Init(&ARGB_InitStructure); // Apply ARGB configuration
    // Enable ARGB interrupt for frame reception complete
    ARGB_ITConfig(ARGB_IT_FRH, ENABLE);
    // Enable ARGB interrupt in the NVIC
    NVIC_EnableIRQ(ARGB_IRQn);
    ARGB_WS2812B_DMA_Init((void *)ARGB_DMA_Buffer, buffer_size);
}

```

需要注意的是：在重新开始一帧传输的时候，需要重新配置 DMA 的传输长度以及传输首地址。

示例

示例将在 8*8 像素数量由 WS2812 组成的点阵屏上显示从 GIF 文件生成的多帧 GRB 数据格式的位图数据。

```

int main(void)
{
    . . .
    GD_ERR err;
    size_t err_pos;
    GD_GIF_HANDLE gif;
    gif = GD_FromMemory(media_default, media_default_end - media_default, &err, &err_pos);
    GD_FRAME *frame = GD_GetFrame(gif, 0);
    ARGB_WS2812B_Init((uint8_t *)frame->Buffer, frame->Descriptor.Width * frame->Descriptor.Height * 3);

    while (1)
    {
        for (uint32_t i = 0; i < GD_FrameCount(gif); i++)
        {
            GD_FRAME *frame = GD_GetFrame(gif, i);
            ARGB_WS2812B_DMA_Init((uint8_t *)frame->Buffer, frame->Descriptor.Width * frame->Descriptor.Height * 3);
            ARGB_Cmd(ENABLE);
            while (DMA1_Channel10->CNTR != 0)
            {
            }
            DMA_Cmd(DMA1_Channel10, DISABLE);
            Delay_Ms(1000);
        }
    }
}

```

2.2 典型应用场景

- 大规模智能照明系统

场景描述：城市景观亮化、建筑轮廓灯、大型舞台背景屏（数百米灯带，数万颗灯珠）。

价值点：利用 DMA + 硬件时序，轻松驱动超长灯带而不丢帧；打造同步且复杂的动态光效。

- 电竞外设与桌面氛围灯

场景描述：机械键盘背光、鼠标灯效、电脑机箱内部幻彩灯条、显示器偏光灯。

价值点：支持 SK6812 (RGBW) 四色驱动，色彩表现更细腻；硬件级低延迟确保灯效能实时响应游戏画面或音乐节奏（如律动模式），无卡顿感。

- 汽车内饰与外部氛围灯

场景描述：车内脚窝灯、门板流水转向灯、格栅动态迎宾灯。

价值点：车规级应用对稳定性要求极高。ARGB 硬件外设消除了软件模拟的不确定性，确保在车辆复杂电磁环境和多任务调度下，灯光效果依然稳定可靠、无闪烁。

- 音乐可视化与互动装置

场景描述：蓝牙音箱律动灯、互动艺术装置、酒吧装饰灯。

价值点：CPU 可全速运行 FFT（快速傅里叶变换）算法解析音频，同时将结果通过 DMA 发送给 ARGB 外设。硬件生成的精准时序保证了光效与音乐的完美同步。

历史版本

更新内容

日期	版本	变更内容
2026/03/26	V1.0	初版发行

声明

本手册版权所有为南京沁恒微电子股份有限公司（Copyright © Nanjing Qinheng Microelectronics Co., Ltd. All Rights Reserved），未经南京沁恒微电子股份有限公司书面许可，任何人不得因任何目的、以任何形式（包括但不限于全部或部分地向任何人复制、泄露或散布）不当使用本产品手册中的任何信息。

任何未经允许擅自更改本产品手册中的内容与南京沁恒微电子股份有限公司无关。

南京沁恒微电子股份有限公司所提供的说明文档只作为相关产品的使用参考，不包含任何对特殊使用目的的担保。南京沁恒微电子股份有限公司保留更改和升级本产品手册以及手册中涉及的产品或软件的权利。

参考手册中可能包含少量由于疏忽造成的错误。已发现的会定期勘误，并在再版中更新和避免出现此类错误。